

Sistemas Distribuídos e Tolerantes a Falhas

Relatório do Trabalho - Parte 2

Nomes:

Lucas Neto Ferreira (00597912)

Jean Smaniotto Argoud (00275602)

(A) Descrição do Ambiente de Testes

Sistema Operacional: Ambiente Unix-like simulado via Windows Subsystem for Linux 2 (WSL2), executando a distribuição Ubuntu 22.04.5 LTS (Kernel 6.6.87.2-microsoft-standard-WSL2).

Hardware Utilizado: Processador Intel i5-1035G1, 8GB de memória RAM.

Interpretador: Python 3.13, fazendo uso exclusivo das primitivas nativas da biblioteca socket (API UDP) e struct para serialização de dados na comunicação inter-processos.

(B) Funcionamento do Algoritmo de Eleição de Líder Implementado

Lógica do Algoritmo (Valentão): O mecanismo de eleição entra em ação de forma dinâmica na inicialização de um nó (TIPO_NOVO_SERVIDOR) ou quando um servidor secundário detecta a ausência de mensagens do líder atual. O processo que inicia a eleição propaga um pacote TIPO_ELEICAO via UDP Broadcast na porta de controle 9999. Servidores ativos com IDs maiores que o do emissor interceptam esse pacote e respondem imediatamente com um TIPO_OK via Unicast, assumindo a liderança do processo eleitoral. Se o processo convocador não for barrado (ou seja, self.fui_barrado permanecer falso) dentro de uma janela de timeout de 0.6 segundos, ele se autodeclara o novo Coordenador (RM Primário) e propaga um pacote TIPO_COORDENADOR para atualizar a topologia da rede.

Justificativa da Escolha: A implementação atende estritamente à especificação do projeto. O algoritmo do Valentão se justifica pela sua alta eficiência em ambientes de rede local (LAN), onde o suporte a pacotes de broadcast permite que a reconfiguração e a convergência do grupo de servidores ocorram de forma ágil, descentralizada e sem a necessidade de manter tabelas estáticas de IPs pré-configuradas.

(C) Implementação da Replicação Passiva

Modelo de Replicação Passiva: O sistema adota o modelo de backup com cópia primária. O cliente interage exclusivamente com o líder na porta 8888. A cada nova operação aritmética processada com sucesso pelo líder (TIPO_REQUISICAO), as variáveis de estado (total_sum e num_reqs) são atualizadas e imediatamente propagadas para os backups através de mensagens de TIPO_REPLICACAO. Esse fluxo também funciona como o *heartbeat* (batimento cardíaco) do sistema, enviado periodicamente a cada 0.3 segundos pelo líder. Se os backups passarem mais de 1.2 segundos sem registrar essa atualização, decretam a falha do líder por timeout e iniciam uma nova eleição de forma automatizada e transparente para o cliente.

Garantia de Sincronização e Concorrência: O servidor foi estruturado para trabalhar de forma multithread. Para processar cada requisição UDP sem bloquear os canais de controle, o servidor dispara threads assíncronas sob demanda (threading.Thread). Para evitar condições de corrida (*race conditions*) provocadas pelo acesso simultâneo ao estado do sistema (como atualizações de estado concorrendo com a leitura de canais de descoberta), utilizou-se a primitiva de exclusão mútua threading.Lock (self.lock) para proteger e isolar o acesso às variáveis globais e ao dicionário de controle de idempotência dos clientes (self.clientes).

(D) Descrição dos Problemas Encontrados e Limitações de Ambiente

Isolamento de Rede no WSL2 e Split-Brain: Durante os testes práticos simulando a execução do sistema em nós distribuídos, constatou-se a ocorrência de um estado de Split-Brain (onde múltiplas instâncias se autodeclaravam líderes simultaneamente). A raiz do problema foi identificada na camada de infraestrutura de rede do Windows Subsystem for Linux (WSL): por padrão, ele opera sob uma arquitetura de switch virtual interno (modo NAT), o que restringe e isola o tráfego de pacotes UDP Broadcast entre a máquina hospedeira e a rede física local. Como os pacotes de supressão TIPO_OK e os anúncios de TIPO_COORDENADOR não cruzavam essa barreira de virtualização, os nós tornavam-se incapazes de convergir para um consenso de liderança unificado.

Abordagem Adotada e Validação de Escopo: Diante das limitações severas de emulação de rede impostas pelo ambiente de desenvolvimento WSL2, e dada a impossibilidade de provisionar hardware com Linux nativo em tempo hábil para os testes inter-máquinas, o escopo de validação do projeto foi delimitado. O algoritmo foi homologado e demonstrou funcionamento lógico 100% correto quando executado localmente (na mesma instância virtual), onde o tráfego de loopback atende aos requisitos do sistema. Dessa forma, assumimos que, em um ambiente operacional composto por servidores Linux nativos interconectados em uma rede local física (LAN), o algoritmo do Valentão e o fluxo de replicação passiva funcionarão conforme especificado, uma vez que as barreiras artificiais de virtualização observadas no WSL2 não estarão presentes.

